

JavaScript Day 3

- Destructuring is a way to unpack arrays, and objects and assigning to a distinct variable.

```
const numbers = [1, 2, 3]
let [numOne, numTwo, numThree] = numbers

console.log(numOne, numTwo, numThree) // 1 2 3
```

Example 2

```
const fullStack = [
  ['HTML', 'CSS', 'JS', 'React'],
  ['Node', 'Express', 'MongoDB']
]
const [frontEnd, backEnd] = fullStack

console.log(frontEnd) // ["HTML", "CSS", "JS", "React"]
console.log(backEnd) ["Node", "Express", "MongoDB"]
```

- If we like to skip one of the values in the array we use additional comma. The comma helps to omit the value at that specific index

```
const numbers = [1, 2, 3]
let [numOne, , numThree] = numbers //2 is omitted

console.log(numOne, numThree)
//1 3
```

- We can use default value in case the value of array for that index is undefined:

```
const names = [undefined, 'Brook', 'David']
let [
  firstPerson = 'Asabeneh',
  secondPerson,
  thirdPerson,
  fourthPerson = 'John'
] = names

console.log(firstPerson, secondPerson, thirdPerson, fourthPerson)
//Asabeneh Brook David John
```

- We can not assign variable to all the elements in the array. We can destructure few of the first and we can get the remaining as array using spread operator(...).

```
const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
let [num1, num2, num3, ...rest] = nums
```

```
console.log(num1, num2, num3)
console.log(rest)
//1 2 3
//[4, 5, 6, 7, 8, 9, 10]
```

- **Destructuring during iteration:**

Example 1:

```
const countries = [['Finland', 'Helsinki'], ['Sweden', 'Stockholm'], ['Norway', 'Oslo']]
```

```
for (const [country, city] of countries) {
  console.log(country, city)
}
//Finland Helsinki
//Sweden Stockholm
//Norway Oslo
```

Example 2:

```
const fullStack = [
  ['HTML', 'CSS', 'JS', 'React'],
  ['Node', 'Express', 'MongoDB']
]
```

```
for(const [first, second, third] of fullStack) {
  console.log(first, second, third)
}
//HTML CSS JS
//Node Express MongoDB
```

- **Destructuring Object**

When we destructure the name of the variable we use to destructure should be exactly the same as the key or property of the object. See the example below.

Example:

```
const rectangle = {
  width: 20,
```

```
    height: 10,  
    area: 200  
  }  
let { width, height, area, perimeter } = rectangle  
  
console.log(width, height, area, perimeter)  
20 10 200 undefined
```

- **Object parameter without destructuring**

```
// Without destructuring  
const rect = {  
  width: 20,  
  height: 10  
}  
const calculatePerimeter = rectangle => {  
  return 2 * (rectangle.width + rectangle.height)  
}  
  
console.log(calculatePerimeter(rect)) // 60
```

- **Object parameter with destructuring**

```
const calculatePerimeter = ({ width, height }) => {  
  return 2 * (width + height)  
}  
  
console.log(calculatePerimeter(rect)) // 60
```

- **Spread operator to copy array:**

```
const evens = [0, 2, 4, 6, 8, 10]  
const evenNumbers = [...evens]  
  
const odds = [1, 3, 5, 7, 9]  
const oddNumbers = [...odds]  
  
const wholeNumbers = [...evens, ...odds]  
  
console.log(evenNumbers)  
console.log(oddNumbers)  
console.log(wholeNumbers)
```

- **Spread operator to copy object:**

We can copy an object using a spread operator

```
const user = {  
  name:'Asabeneh',  
  title:'Programmer',  
  country:'Finland',  
  city:'Helsinki'  
}
```

```
const copiedUser = {...user}  
console.log(copiedUser)  
{name: "Asabeneh", title: "Programmer", country: "Finland", city: "Helsinki"}
```

Modifying or changing the object while copying

```
const user = {  
  name:'Asabeneh',  
  title:'Programmer',  
  country:'Finland',  
  city:'Helsinki'  
}
```

```
const copiedUser = {...user, title:'instructor'}  
console.log(copiedUser)  
{name: "Asabeneh", title: "instructor", country: "Finland", city: "Helsinki"}
```

- **RegEx:**

https://github.com/Asabeneh/30-Days-Of-JavaScript/blob/master/12_Day_Regular_expressions/12_day_regular_expressions.md

Study from the above link, is used to match patterns in text. Have multiple methods such as replace, test, match. Have flags.

g: a global flag which means looking for a pattern in whole text

i: case insensitive flag(it searches for both lowercase and uppercase)

m: multiline

Defined as js: let pattern = /python/g;

Json:

Json to object and object to json:

Mostly we fetch JSON data from HTTP response or from a file, but we can store the JSON as a string and we can change to Object for sake of demonstration. In JavaScript the keyword JSON has `parse()` and `stringify()` methods. When we want to change the JSON to an object we parse the JSON using `JSON.parse()`. When we want to change the object to JSON we use `JSON.stringify()`.

Error Handling `try`, `catch`, `finally` and `throw`.

Console Methods such as `console.error`, `console.warn`, [console.info](#), `console.table`, `console.time` and `console.endtime()`.

Web Storage (`sessionStorage` and `localStorage`) is a new HTML5 API offering important benefits over traditional cookies. Before HTML5, application data had to be stored in cookies, included in every server request. Web storage is more secure, and large amounts of data can be stored locally, without affecting website performance. The data storage limit of cookies in many web browsers is about 4 KB per cookie. Web Storages can store far larger data (at least 5MB) and never transferred to the server. All sites from the same or one origin can store and access the same data.

The data being stored can be accessed using JavaScript, which gives you the ability to leverage client-side scripting to do many things that have traditionally involved server-side programming and relational databases. There are two Web Storage objects:

`sessionStorage`

`localStorage`

`localStorage` is similar to `sessionStorage`, except that while data stored in `localStorage` has no expiration time, data stored in `sessionStorage` gets cleared when the page session ends — that is, when the page is closed.

It should be noted that data stored in either `localStorage` or `sessionStorage` is specific to the protocol of the page.

Use case of Web Storages

Some use case of Web Storages are

- store data temporarily
- saving products that the user places in his shopping cart
- data can be made available between page requests, multiple browser tabs, and also between browser sessions using `localStorage`

- can be used offline completely using localStorage
- Web Storage can be a great performance win when some static data is stored on the client to minimize the number of subsequent requests. Even images can be stored in strings using Base64 encoding.
- can be used for user authentication method

Storing string in a localStorage:

Example1

localStorage.setItem('firstName', 'Asabeneh') // since the value is string we do not stringify it

console.log(localStorage)

Storage {firstName: 'Asabeneh', length: 1}

Example2

Storing number in a local storage

localStorage.setItem('age', 200)

console.log(localStorage)

Storage {age: '200', firstName: 'Asabeneh', length: 2}

Storing an array in a localStorage. If we are storing an array, an object or object array, we should stringify the object first. See the example below.

const skills = ['HTML', 'CSS', 'JS', 'React']

//Skills array has to be stringified first to keep the format.

const skillsJSON = JSON.stringify(skills, undefined, 4)

localStorage.setItem('skills', skillsJSON)

console.log(localStorage)

Storage {age: '200', firstName: 'Asabeneh', skills: 'HTML,CSS,JS,React', length: 3}

let skills = [

 { tech: 'HTML', level: 10 },

 { tech: 'CSS', level: 9 },

 { tech: 'JS', level: 8 },

 { tech: 'React', level: 9 },

 { tech: 'Redux', level: 10 },

 { tech: 'Node', level: 8 },

 { tech: 'MongoDB', level: 8 }]

let skillJSON = JSON.stringify(skills)

localStorage.setItem('skills', skillJSON)

Storing an object in a localStorage. Before we storage objects to a localStorage, the object has to be stringified.

const user = {

```
  firstName: 'Asabeneh',  
  age: 250,  
  skills: ['HTML', 'CSS', 'JS', 'React']  
}
```

```
const userText = JSON.stringify(user, undefined, 4)  
localStorage.setItem('user', userText)
```

Getting item from localStorage

We get data from the local storage using `localStorage.getItem()` method.

```
//syntax  
localStorage.getItem('key')  
let firstName = localStorage.getItem('firstName')  
let age = localStorage.getItem('age')  
let skills = localStorage.getItem('skills')  
console.log(firstName, age, skills)  
'Asabeneh', '200', ['HTML', 'CSS', 'JS', 'React']
```

As you can see the skill is in a string format. Let us use `JSON.parse()` to parse it to normal array.

```
let skills = localStorage.getItem('skills')  
let skillsObj = JSON.parse(skills, undefined, 4)  
console.log(skillsObj)  
['HTML', 'CSS', 'JS', 'React']
```

Clearing the localStorage

The `clear` method, will clear everything stored in the local storage

```
localStorage.clear()
```

Classes:

- Objects can inherit properties of other objects through prototype. For example

```
const employee = {  
  calcTax() {  
    console.log("tax rate is 10%");  
  }  
};  
  
const karan = {  
  salary: 50000
```

```
};  
  
karan.__proto__ = employee;
```

Here karan is able to call calcTax as it has a prototype of employee.

If object and prototype have same method object method will be called by object

- Classes: blueprint for objects, contains methods and variables. Also have constructor which invokes on its own. Constructor can be parametrized as well.

```
class ToyotaCar{  
  constructor(brand) {  
    console.log("This is a toyota car");  
    this.brand = brand;  
  }  
  modelName() {  
    console.log("the model name is ", this.brand);  
  }  
}  
  
let fortuner = new ToyotaCar("fortuner");  
console.log(fortuner);
```

- For inheritance “extends” keyword is used

```
class Parent{  
  hello() {  
    console.log("Hello");  
  }  
}  
  
class Child extends Parent {  
  
}  
  
let obj = new Child();  
obj.hello();
```

If child and parent have same method name child's method will be invoked by child object this is called method overriding.

super() keyword written in derived class constructor to invoke the constructor of parent class. Similarly any parameter written in super() will also be passed to the

parent class constructor parameter. Also to access the methods of parent class in child class method we access by `super.eat()`.

Asynchronous vs synchronous:

- Synchronous executes code in serial way line by line execution.
- Asynchronous makes some line execute in background and move onto execute next such as server calls or api or data fetching can be asynchronous.

Callback Hell:

- Nested callbacks making a pyramid like structure like one callback has another callback in it which contains another callback in it. This type of code becomes unable to understand and comprehend.

Promises:

- Promises came to solve callback hell.
- Promises is for eventual completion of task. It is an object in JS.
- Promise has three states pending, fulfilled(resolved) and rejected.
- Lets say there is a data api which gives us data, when we request for data it gives us a promise that it will give data and promise state is pending and say after 5 secs it gives us actual data with promise state resolved.
- Usually we don't create promises we get them in return through api's.
- Lets say we get a promise, now if the promise is fulfilled we do things by `promise.then(callback function)` and if the promise is rejected we use `promise.catch(err)`

```
const getPromise = () => {
  return new Promise ((resolve, reject) => {
    console.log("I am a promise");
    resolve("success");
  })
};

let promise = getPromise();
promise.then( (res) => {
  console.log("promise fulfilled" , res);
});

promise.catch((err) => {
  console.log("rejected", err);
})
```

- Chaining of promises: Now consider there are two data sources and both will be returning promises when we call their api so in above code there are two different function returning two different promises now we want to get data from one promise first after getting its data we want to get data from other promise so we use `promise.then()` inside of `first promise.then()` such that when first promise is fulfilled the second promise is then getting data.

Async-Await:

- `async` is a keyword when used to make a function makes it compulsory for the function to return a promise.
 - `await` pauses the surrounding asynchronous function until its function promise is settled.
 - `await` is used inside an `async` function.
 - IIFE can be used here as we need a function only to use `await` call inside an `async` function and then we have to call that function. By IIFE the function is immediately invoked as created `(func)()` by `func` we mean full func body.
-

What is the Event Loop?

- JavaScript is **single-threaded**: only one task runs at a time on the **call stack**.
 - The **event loop** allows JavaScript to handle **asynchronous operations** (e.g., timers, network calls) **without blocking** the main thread.
 - It enables **non-blocking I/O**, giving the illusion of multi-threading.
-

Key Components of the Event Loop

1. Call Stack

- Holds function calls.
- **Last In, First Out (LIFO)**.
- Executes synchronous code line-by-line.

2. Web APIs / Browser APIs

- Provided by the browser or Node.js environment.
- Handles async tasks like:
 - `setTimeout()`
 - `fetch()`
 - `addEventListener()`
 - DOM events
- After task completion, results are sent to the **callback queue**.

3. Callback Queue / Task Queue

- Holds **callbacks** from completed async tasks.
- Functions here **wait** until the call stack is empty.

4. Microtask Queue

- Contains **Promises** and `async/await` callbacks.
- Has **higher priority** than the callback queue (macro-task queue).
- Runs **before** tasks in the callback queue.

5. Event Loop

- Continuously checks:
 - Is the **call stack empty**?
 - Are there any **microtasks** or **callbacks** waiting?

- If so, it moves them to the call stack for execution.

How It Works (Step-by-Step)

Example:

javascript

CopyEdit

```
console.log('Start');
```

```
setTimeout(() => {  
  console.log('Inside timeout');  
}, 0);
```

```
Promise.resolve().then(() => {  
  console.log('Inside Promise');  
});
```

```
console.log('End');
```

Execution Flow:

1. `console.log('Start')` → runs immediately (call stack)
2. `setTimeout()` → sent to **Web API**, scheduled for 0ms
3. `Promise.then()` → added to **Microtask Queue**
4. `console.log('End')` → runs immediately
5. Call Stack is now empty → Event Loop starts:
 - First runs **microtasks** → `console.log('Inside Promise')`
 - Then runs **callback queue** → `console.log('Inside timeout')`

Output:

scss

CopyEdit

Start

End

Inside Promise

Inside timeout

 Important Rules

- **Microtask queue (Promises)** always runs **before** the callback queue (`setTimeout`, etc.).
- Event loop runs **forever** (until the program exits).
- Long-running synchronous code **blocks the call stack**, delaying async tasks.